

전자전기컴퓨터설계실험Ⅱ (09)

<final project - 전자시계>

결과레포트



지도교수: 김민준 교수님

전자전기컴퓨터공학부

박소연(2024440057)

실험 날짜: 2025.12.12

목차

1. 초록(Abstact).....	1
2. 서론(Introduction).....	1
a. 실험 목표.....	1
b. 시스템 개요.....	1
3. 시스템 구성 및 설계	1
a. 전체 블록 다이어그램	1
b. Top Module 및 FSM 설계	2
c. 세부 기능 구현 계획.....	2
4. 상세 기능 구현.....	2
a. 전자 시계.....	2
5. 구현 결과(Implementation Results)	8
a. 준비물.....	8
b. 기능변 동작 사진.....	8
6. 결론 및 고찰(Conclusion & Discssion)	11
a. 설계 요약.....	11
b. 문제점 및 해결 과정(troubleshooting).....	11
c. 개선 사항 및 제언	12
7. 부록(Appendix)	12
8. 참고문헌(Reference).....	13

1. 초록(Abstract)

본 프로젝트는 Verilog HDL을 이용하여 다양한 부가 기능을 갖춘 디지털 시계 시스템을 설계하고 구현하는 것을 목표로 한다. 시스템은 Top-Down 방식을 적용하여 최상위 모듈(top_system) 아래에 시계, 스톱워치, 타이머, 알람, 세계 시간, 날짜 등 각 기능별 하위 모듈을 인스턴스화하여 구성하였다. 전체 시스템의 동작은 유한 상태 머신(FSM)을 기반으로 설계된 메뉴(menu.v) 모듈에 의해 제어되며, 사용자는 키패드와 스위치를 통해 모드 간 이동 및 기능 제어를 수행할 수 있다. 출력 인터페이스로는 7-세그먼트(7-Segment)를 사용하여 시간 및 숫자 데이터를 표시하고, Text LCD를 통해 현재 모드와 동작 상태 정보를 시각적으로 제공하였다. 특히, 피에조(Piezo) 부저를 활용한 청각적 알람 기능과 UTC(협정 세계시) 변환 알고리즘을 적용한 세계 시간(World Time) 기능을 구현하여 시스템의 완성도를 높였다. 최종적으로 시뮬레이션 및 FPGA 보드 검증을 통해 모든 기능이 설계 사양에 맞춰 정확히 동작함을 확인하였다.

2. 서론(Introduction)

a. 실험 목표

Verilog HDL을 이용하여 FSM 기반의 모드 제어 로직을 설계하고, 7-Segment, Text LCD, Piezo 부저 등 다양한 입출력 장치를 활용하여 시계, 스톱워치, 알람, 세계 시간 등의 기능이 통합된 디지털 시계 시스템을 구현한다.

b. 시스템 개요

본 프로젝트는 Xilinx Vivado 설계 환경에서 Verilog HDL을 이용하여 구현된 FPGA 기반의 다기능 디지털 시계 시스템이다. 전체 시스템은 50MHz의 시스템 클럭을 기반으로 동작하며, 시간 계수 및 타이밍 제어를 위해 내부적으로 생성된 1kHz 클럭 신호를 사용한다.

시스템은 크게 사용자 조작을 위한 입력 인터페이스, 시스템의 상태 관리 및 연산을 수행하는 제어 유닛(Control Unit), 그리고 처리 결과를 사용자에게 전달하는 출력 인터페이스로 구성된다. 입력부는 메뉴 탐색 및 모드 변경을 위한 2개의 푸시 버튼과 기능 선택 및 데이터 설정을 위한 11개의 슬라이드 스위치로 구성되어 직관적인 사용자 경험(UX)을 제공한다.

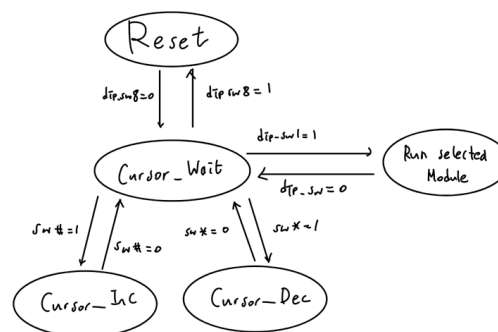
시스템의 핵심인 제어부는 유한 상태 머신(FSM)을 기반으로 설계되어 시계, 알람, 스톱워치와 같은 기본 기능뿐만 아니라 반응 속도 게임, 피아노 연주 등 총 14가지의 다양한 동작 모드를 통합 관리한다. 또한, 각 모듈 간의 신호 충돌을 방지하고 안정적인 데이터 흐름을 보장하기 위해 출력단에는 멀티플렉서(MUX) 구조를 적용하였다.

출력 인터페이스는 시분할 제어(Time-Division Multiplexing)

방식의 7-Segment와 텍스트 정보를 표시하는 LCD를 메인 디스플레이로 활용하며, LED, 피에조 부저, 서보 모터를 부가적으로 제어하여 시각적, 청각적, 물리적 피드백이 결합된 입체적인 시스템을 구현하였다.

3. 시스템 구성 및 설계

a. 전체 블록 다이어그램

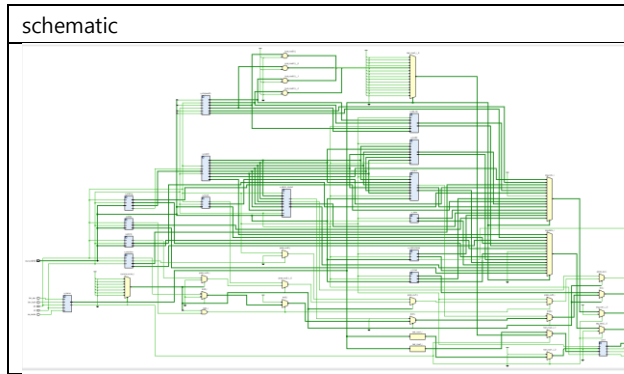


제안된 시스템의 동작은 Reset, Cursor_Wait, Cursor_Inc, Cursor_Dec, Run_Selected_Module의 5가지 상태를 갖는 유한 상태 머신(FSM)에 의해 제어된다.

시스템에 전원이 인가되거나 리셋 스위치(dip_sw8)가 High(1) 상태가 되면, 시스템은 초기 상태인 Reset 상태에 진입하여 모든 레지스터와 내부 신호를 초기화한다. 이후 리셋 신호가 해제(dip_sw8=0)되면 시스템은 대기 상태인 Cursor_Wait 상태로 전이한다. 이 상태는 사용자가 기능을 탐색하고 선택하는 단계로, LCD에 현재 커서가 가리키는 모드 이름을 표시하며 입력을 대기한다.

Cursor_Wait 상태에서 우측 버튼(sw#)을 누르면 Cursor_Inc 상태로 전이하여 커서 값을 증가시키고, 버튼 입력이 해제되면 다시 대기 상태로 복귀한다. 반대로 좌측 버튼(sw*)을 누르면 Cursor_Dec 상태로 전이하여 커서 값을 감소시킨다. 사용자는 이 과정을 통해 시계, 스톱워치, 게임, 피아노 등 총 14가지의 다양한 기능 모드를 순차적으로 탐색할 수 있다.

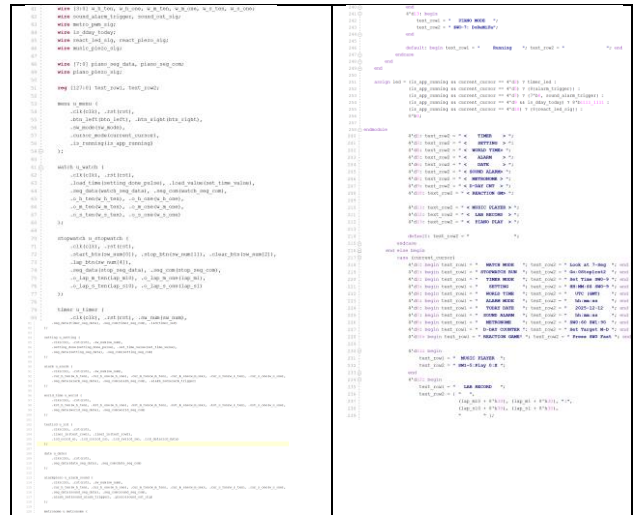
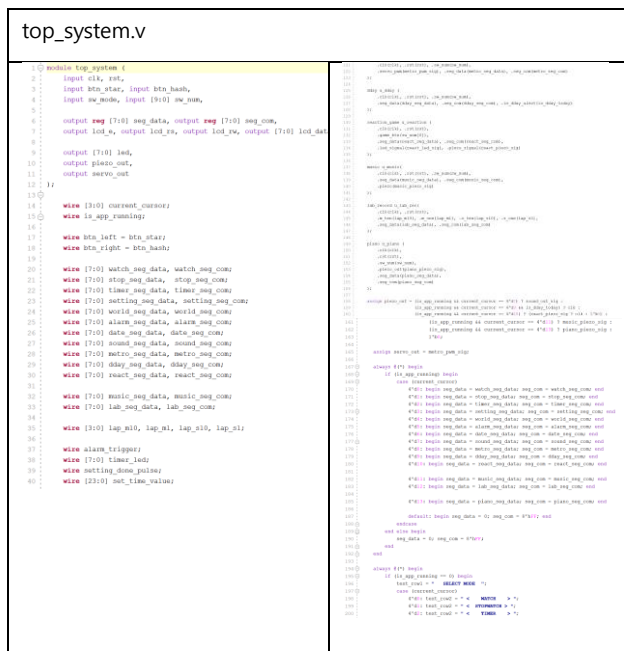
원하는 기능이 선택된 상태에서 모드 선택 스위치(dip_sw1)를 High(1)로 설정하면, FSM은 Run_Selected_Module 상태로 전이하여 해당 기능을 실행한다. 이 상태에서는 선택된 모듈(Module)이 활성화되어 7-Segment, LED, 피에조 부저 등을 통해 동작 결과를 실시간으로 출력한다. 다시 스위치를 Low(0)로 전환하면 실행을 중단하고 Cursor_Wait 상태로 복귀하여 다른 기능을 선택할 수 있도록 설계되었다.



b. Top Module 및 FSM 설계

Top Module은 top_system으로 정의하며, 하위 모듈들을 인스턴스화(Instantiation)하고 모듈 간의 신호 흐름을 제어하는 최상위 계층이다. 이 모듈은 FPGA의 시스템 클럭(50MHz)을 입력 받아 각 기능 모듈에 분배(Clock Distribution)하여 시스템의 동기화된 동작을 보장한다. 시계나 스톱워치와 같이 시간 계수가 필요한 하위 모듈들은 이 클럭을 기반으로 내부 카운터를 통해 1kHz, 1Hz 등의 타이밍 신호를 자체적으로 생성하여 동작한다.

또한, 사용자의 버튼 입력에 따라 현재 동작 모드(State)를 결정하고, 시계나 스톱워치, 피아노 등 각 하위 모듈에서 출력되는 데이터 중 현재 모드에 적합한 데이터를 선별하여 디스플레이 장치로 라우팅하는 모드 제어(Mode Muxing) 기능을 담당한다. 마지막으로 버튼과 스위치 입력을 각 하위 모듈로 전달하여 엣지 검출(Edge Detection)을 가능하게 하고, 연산된 결과 데이터를 7-Segment, LED, LCD 등의 출력 타이밍에 맞춰 안정적으로 내보내는 입출력 인터페이스 제어 역할을 통합적으로 수행한다.



c. 세부 기능 구현 계획

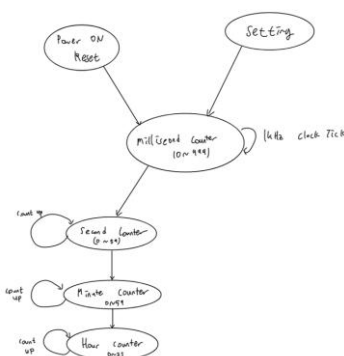
주파수 분주 설계는 FPGA의 시스템 클럭인 50MHz를 입력 받아 내부 카운터를 통해 각 모듈이 요구하는 동작 주파수로 변환하는 것을 목표로 한다. 우선 피아노 모듈과 같은 고속 동작을 위해 50MHz 원본 클럭을 사용하며, 시계 및 스톱워치의 시간 계수를 위해 이를 분주하여 1kHz(1ms)의 기준 클럭 신호를 생성한다.

각 시간 관련 모듈(Watch, Stopwatch 등)은 이 1kHz 클럭을 받아 내부적으로 1000을 카운트하여 정확한 1Hz(1초) 신호를 생성하는 방식을 채택하였다. 또한, 7-Segment 디스플레이의 동적 구동(Dynamic Scanning)을 위해 약 200Hz 대역의 스캐닝 주파수를 적용함으로써, 사람의 눈에 잔상 효과를 일으켜 플리커(Flicker) 현상 없이 안정적인 출력을 제공하도록 설계되었다.

4. 상세 기능 구현

a. 전자 시계

1. 전자 시계



본 프로젝트의 핵심 기능인 디지털 시계 모듈(watch.v)은 1kHz의 시스템 클럭을 기반으로 시간의 흐름을 계수하고 이를 디스플레이에 출력하는 역할을 수행한다. 이 모듈은 크게 시간 계수 로직(Time Counter Logic)과 디스플레이 디코딩 로직

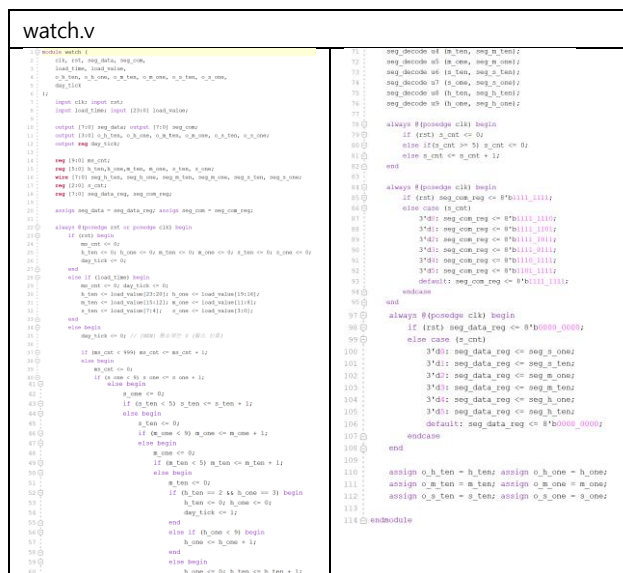
(Display Decoding Logic)으로 구성된다.

시간 계수 로직은 1ms 단위의 틱(Tick)을 생성하는 ms_cnt 레지스터를 시작으로, 하위 카운터의 캐리(Carry)가 상위 카운터를 트리거하는 계층적 구조로 설계되었다. ms_cnt가 0부터 999까지 증가하여 1초가 경과하면 s_one(초의 1의 자리)이 증가하고, BCD(Binary Coded Decimal) 방식의 연쇄적인 카운팅을 통해 초, 분, 시가 순차적으로 증가한다.

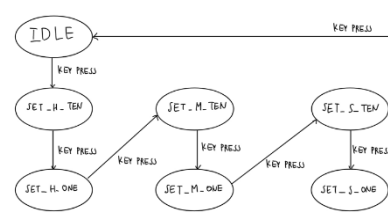
카운터는 60진법(초, 분)과 24진법(시) 논리에 따라 동작하며, 23시 59분 59초를 넘기는 순간 00시 00분 00초로 자동 초기화되어 순환하도록 구현되었다. 특히 이 시점에 하루가 경과했음을 알리는 day_tick 펄스 신호를 생성하여, 외부 날짜 모듈이 자동으로 일(Day)을 증가시킬 수 있도록 모듈 간 동기화 로직을 추가하였다.

또한, 외부의 시간 설정 모듈(setting.v)로부터 load_time 신호가 입력될 경우, 현재 카운팅 중인 값을 사용자가 설정한 load_value로 즉시 덮어쓰는 동기식 로드(Synchronous Load) 기능을 포함하여 시간 조정의 편의성을 확보하였다.

디스플레이 출력부는 6자리의 시간 데이터(시, 분, 초)를 7-Segment에 표시하기 위해 다이내믹 스캐닝(Dynamic Scanning) 기법을 사용한다. s_cnt라는 3비트 카운터를 이용하여 약 166Hz의 속도로 6개의 자릿수 선택 신호(seg_com)를 순차적으로 활성화하고, 해당 타이밍에 맞춰 현재 자릿수에 해당하는 숫자 데이터(seg_data)를 출력함으로써 잔상 효과를 통해 사용자가 끊임 없는 숫자로 인식하게끔 설계하였다.

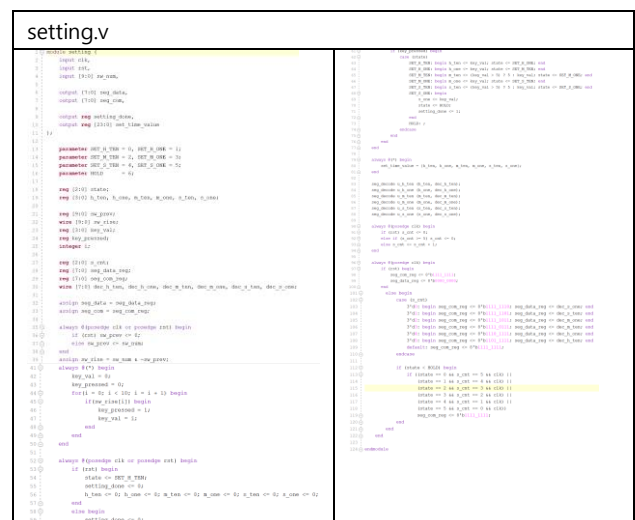


2. 부가기능 - 시간 설정

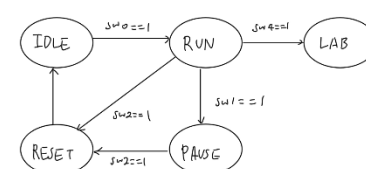


시간 설정 모
듈(setting.v)은
시스템 시각 변
경을 위한 사용
자 인터페이스를
제공한다. 이 모

물은 내부적으로 `set_time_value`라는 24비트 버퍼 레지스터를 운용하여 설정 데이터를 저장하며, 전체 동작은 유한 상태 머신(FSM)에 의해 제어된다. 설정 모드 진입 시 FSM은 시(Hour)의 10의 자리 설정 단계부터 시작하며, 스위치(`sw_num`) 입력이 감지될 때마다 해당 자릿수의 값을 갱신하고 시의 1의 자리, 분, 초 순으로 상태를 순차 천이시킨다. 모든 자릿수의 설정이 완료되면 `setting_done` 트리거 펄스를 생성하여 메인 시계 모듈(`watch.v`)로 전송한다. 시계 모듈은 이 신호를 감지하는 즉시 카운팅을 일시 중단하고, `load_value` 포트를 통해 전달된 24비트 설정값을 내부 카운터 레지스터에 병렬 로드(Parallel Load)하여 시간을 동기화한다. 이러한 구조는 설정 중인 임시 데이터가 실제 계수 중인 시간에 영향을 주지 않도록 격리하여 시스템의 안정성을 보장한다.



3. Stopwatch



스톱워치 모듈
(stopwatch.v)은 1ms
단위의 정밀한 시간 계
수를 수행하는 타이머
시스템이다. 본 기능은

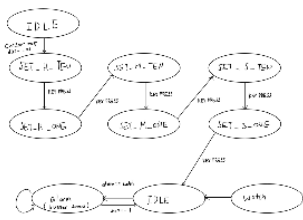
시스템의 안정적인 제어를 위해 RESET(초기화), IDLE(대기), RUN(계수), PAUSE(일시정지), LAB(랩타임)의 5단계 상태를 갖는 유한 상태 머신(FSM) 알고리즘을 기반으로 동작한다. 리셋 신호(SW2)가 입력되면 RESET 상태로 진입하여 레지스터를 초기화한 후 IDLE 상태에서 입력을 대기한다. 시작 버튼(SW0) 인가 시 RUN 상태로 전이하여 1ms 단위로 시간을 적산하며, 정지

버튼(SW1) 입력 시 PAUSE 상태로 전환하여 계수를 일시 정지한다. 특히, RUN 상태에서 랩 타임 버튼(SW4)을 누르면 현재의 카운터 값을 래치(Latch)하여 별도의 레지스터에 저장하는 LAB 동작을 수행한다. 이때 메인 카운터는 백그라운드에서 연속적으로 동작하며, 저장된 랩 타임 데이터는 랩 기록 확인 모듈로 전달된다. 모든 시간 데이터는 각 자릿수별로 독립적인 레지스터(10의 자리, 1의 자리)로 관리되어 별도의 이진-BCD 변환 과정 없이 디스플레이에 즉시 출력 가능하다.

stopwatch.v	
<pre> 1 module stopwatch (2 input clk, 3 input rst, 4 input start_btn, 5 input stop_btn, 6 input clear_btn, 7 input lap_btn, 8 9 output [7:0] seg_data, 10 output [7:0] seg_com, 11 12 output [3:0] o_lap_m_ten, 13 output [3:0] o_lap_m_one, 14 output [3:0] o_lap_s_ten, 15 output [3:0] o_lap_s_one 16); 17 18 reg is_running; 19 reg [10:0] seg_cnt; 20 reg [10:0] m_ten, m_one, s_ten, s_one; 21 reg [10:0] lap_m_ten, lap_m_one, lap_s_ten, lap_s_one; 22 23 wire [7:0] seg_m_ten, seg_m_one, seg_s_ten, seg_s_one; 24 25 reg [1:0] s_cnt; 26 reg [7:0] seg_data_req; 27 reg [7:0] seg_com_req; 28 29 assign seg_data = seg_data_req; 30 assign seg_com = seg_com_req; 31 32 always @(posedge clk or posedge rst) begin 33 if (rst) begin 34 is_running <= 0; 35 else if (clear_btn) begin 36 is_running <= 0; 37 end 38 else if (start_btn) begin 39 is_running <= 1; 40 end 41 else if (lap_btn) begin 42 is_running <= 1; 43 end 44 end 45 46 reg lap_btn_prev; 47 wire lap_btn_rise = lap_btn & ~lap_btn_prev; 48 49 always @(posedge clk) begin 50 if (rst) lap_btn_prev <= 0; 51 else lap_btn_prev <= lap_btn; 52 end 53 54 always @(posedge clk or posedge rst) begin 55 if (rst) begin 56 lap_m_ten <= 0; 57 lap_m_one <= 0; 58 lap_s_ten <= 0; 59 lap_s_one <= 0; 60 end 61 else if (lap_btn_rise) begin 62 lap_m_ten <= m_ten; 63 lap_m_one <= m_one; 64 lap_s_ten <= s_ten; 65 lap_s_one <= s_one; 66 end 67 end 68 69 always @(posedge clk or posedge rst) begin 70 if (rst) begin 71 seg_cnt <= 0; 72 m_ten <= 0; 73 m_one <= 0; 74 s_ten <= 0; 75 s_one <= 0; 76 end 77 else if (is_running) begin 78 seg_cnt <= seg_cnt + 1; 79 if (seg_cnt == 9999) begin 80 m_ten <= m_ten + 1; 81 if (m_ten == 59) begin 82 s_ten <= s_ten + 1; 83 if (s_ten == 59) begin 84 seg_cnt <= 0; 85 m_ten <= m_ten + 1; 86 if (m_ten == 10) begin 87 s_ten <= 0; 88 m_one <= m_one + 1; 89 if (m_one == 10) begin 90 seg_cnt <= 0; 91 m_ten <= 0; 92 s_one <= s_one + 1; 93 if (s_one == 10) begin 94 seg_cnt <= 0; 95 m_ten <= 0; 96 m_one <= 0; 97 seg_cnt <= 0; 98 end 99 end 100 end 101 end 102 end 103 end 104 end 105 end 106 107 always @(posedge clk) begin 108 if (rst) seg_data_req <= 8'b1111_1111; 109 else case (s_cnt) 110 0: seg_data_req <= 8'b1111_1111; 111 1: seg_data_req <= 8'b1111_1111; 112 2: seg_data_req <= 8'b1111_1111; 113 3: seg_data_req <= 8'b1111_1111; 114 4: seg_data_req <= 8'b1111_1111; 115 5: seg_data_req <= 8'b1111_1111; 116 6: seg_data_req <= 8'b1111_1111; 117 7: seg_data_req <= 8'b1111_1111; 118 8: seg_data_req <= 8'b1111_1111; 119 9: seg_data_req <= 8'b1111_1111; 120 end 121 end 122 123 always @(posedge clk) begin 124 if (rst) seg_data_req <= 8'b0000_0000; 125 else case (s_cnt) 126 0: seg_data_req <= 8'b0000_0000; 127 1: seg_data_req <= 8'b0000_0000; 128 2: seg_data_req <= 8'b0000_0000; 129 3: seg_data_req <= 8'b0000_0000; 130 4: seg_data_req <= 8'b0000_0000; 131 5: seg_data_req <= 8'b0000_0000; 132 6: seg_data_req <= 8'b0000_0000; 133 7: seg_data_req <= 8'b0000_0000; 134 8: seg_data_req <= 8'b0000_0000; 135 9: seg_data_req <= 8'b0000_0000; 136 end 137 end 138 139 assign o_lap_m_ten = lap_m_ten; 140 assign o_lap_m_one = lap_m_one; 141 assign o_lap_s_ten = lap_s_ten; 142 assign o_lap_s_one = lap_s_one; 143 endmodule </pre>	<pre> 100 always @(posedge clk) begin 101 if (rst) seg_com_req <= 8'b0000_0000; 102 else case (s_cnt) 103 0: seg_com_req <= 8'b0000_0000; 104 1: seg_com_req <= 8'b0000_0000; 105 2: seg_com_req <= 8'b0000_0000; 106 3: seg_com_req <= 8'b0000_0000; 107 4: seg_com_req <= 8'b0000_0000; 108 5: seg_com_req <= 8'b0000_0000; 109 6: seg_com_req <= 8'b0000_0000; 110 7: seg_com_req <= 8'b0000_0000; 111 8: seg_com_req <= 8'b0000_0000; 112 9: seg_com_req <= 8'b0000_0000; 113 end 114 end 115 endmodule </pre>

4. LAB 확인기능

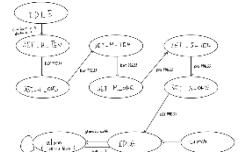
랩 타임(Lab Time) 기능은 데이터의 캡처(Capture)와 확인(Viewer) 단계가 분리된 구조로 설계되었다. 스톱워치가 RUN 상태일 때 랩 타임 버튼(SW4)이 입력되면, 시스템은 0.5초의 시각 데이터를 즉시 래치(Latch)하여 전용 레지스터에 저장한다. 이때 스톱워치 화면은 계속해서 흐르는 시간을 표시하지만, 사용자가 메뉴를 조작하여 랩 기록(Lab Record) 모드로 진입하면 디스플레이 MUX는 저장해 두었던 정적 레지스터 값을 출력한다. 내부적으로는 메인 타이머가 백그라운드에서 끊임없이 동작하고 있으므로, 기록 확인 후 다시 스톱워치 모드로 복귀하더라도 시간의 누락 없이 정확한 총 경과 시간을 이어서 확인할 수 있다.



lab_record.v	
<pre> 1 module lab_record (2 input clk, 3 input rst, 4 input [3:0] m_ten, input [3:0] m_one, 5 input [3:0] s_ten, input [3:0] s_one, 6 7 output [7:0] seg_data, 8 output [7:0] seg_com, 9 10 output [3:0] o_lab_m_ten, 11 output [3:0] o_lab_m_one, 12 output [3:0] o_lab_s_ten, 13 output [3:0] o_lab_s_one 14); 15 16 wire [7:0] seg_m_ten, seg_m_one, seg_s_ten, seg_s_one; 17 18 reg [10:0] seg_cnt; 19 reg [10:0] m_ten, m_one, s_ten, s_one; 20 reg [10:0] lab_m_ten, lab_m_one, lab_s_ten, lab_s_one; 21 22 wire [7:0] seg_data_req; 23 wire [7:0] seg_com_req; 24 25 assign seg_data = seg_data_req; 26 assign seg_com = seg_com_req; 27 28 always @(posedge clk or posedge rst) begin 29 if (rst) begin 30 seg_cnt <= 0; 31 m_ten <= 0; 32 m_one <= 0; 33 s_ten <= 0; 34 s_one <= 0; 35 end 36 else if (start_btn) begin 37 seg_cnt <= seg_cnt + 1; 38 if (seg_cnt == 9999) begin 39 m_ten <= m_ten + 1; 40 if (m_ten == 59) begin 41 s_ten <= s_ten + 1; 42 if (s_ten == 59) begin 43 seg_cnt <= 0; 44 m_ten <= m_ten + 1; 45 if (m_ten == 10) begin 46 s_ten <= 0; 47 m_one <= m_one + 1; 48 if (m_one == 10) begin 49 seg_cnt <= 0; 50 m_ten <= 0; 51 s_one <= s_one + 1; 52 if (s_one == 10) begin 53 seg_cnt <= 0; 54 m_ten <= 0; 55 m_one <= 0; 56 seg_cnt <= 0; 57 end 58 end 59 end 60 end 61 end 62 end 63 end 64 end 65 66 always @(posedge clk) begin 67 if (rst) seg_data_req <= 8'b1111_1111; 68 else case (s_cnt) 69 0: seg_data_req <= 8'b1111_1111; 70 1: seg_data_req <= 8'b1111_1111; 71 2: seg_data_req <= 8'b1111_1111; 72 3: seg_data_req <= 8'b1111_1111; 73 4: seg_data_req <= 8'b1111_1111; 74 5: seg_data_req <= 8'b1111_1111; 75 6: seg_data_req <= 8'b1111_1111; 76 7: seg_data_req <= 8'b1111_1111; 77 8: seg_data_req <= 8'b1111_1111; 78 9: seg_data_req <= 8'b1111_1111; 79 end 80 end 81 82 always @(posedge clk) begin 83 if (rst) seg_data_req <= 8'b0000_0000; 84 else case (s_cnt) 85 0: seg_data_req <= 8'b0000_0000; 86 1: seg_data_req <= 8'b0000_0000; 87 2: seg_data_req <= 8'b0000_0000; 88 3: seg_data_req <= 8'b0000_0000; 89 4: seg_data_req <= 8'b0000_0000; 90 5: seg_data_req <= 8'b0000_0000; 91 6: seg_data_req <= 8'b0000_0000; 92 7: seg_data_req <= 8'b0000_0000; 93 8: seg_data_req <= 8'b0000_0000; 94 9: seg_data_req <= 8'b0000_0000; 95 end 96 end 97 98 assign o_lab_m_ten = lab_m_ten; 99 assign o_lab_m_one = lab_m_one; 100 assign o_lab_s_ten = lab_s_ten; 101 assign o_lab_s_one = lab_s_one; 102 endmodule </pre>	<pre> 39 always @(posedge clk) begin 40 if (rst) seg_data_req <= 0; 41 else case (scan_cnt) 42 0: seg_data_req <= seg_m_ten; 43 1: seg_data_req <= seg_m_one; 44 2: seg_data_req <= seg_s_ten; 45 3: seg_data_req <= seg_s_one; 46 end 47 end 48 end 49 endmodule </pre>

5. 알람

알람 모듈(alarm.v)은 사용자가 지정한 시각에 도달하면 시각적 신호를 발생시킨다. 설정 방식은 시간 설정 모듈과 동일하게 시, 분, 초를 순차적으로 입력 받아 내부 레지스터에 저장한다. 시스템은 매 클럭마다 현재 시계 모듈(watch.v)의 시, 분, 초 데이터와 설정된 알람 시각을 비교기(Comparator)를 통해 실시간 대조한다. 두 값이 정확히 일치하는 순간 알람 활성화 플레그가 셋(Set)되어 LED를 점등시킨다. 이 신호는 특정 시간 동안만 유지되는 것이 아니라 래치(Latch) 구조로 설계되어, 사용자가 해제 스위치(SW0)를 누르기 전까지 지속적으로 알람 상태를 유지하도록 구현되었다.



alarm.v	
<pre> 1 module alarm (2 input clk, 3 input rst, 4 input [3:0] m_ten, input [3:0] m_one, 5 input [3:0] s_ten, input [3:0] s_one, 6 7 output [7:0] seg_data, 8 output [7:0] seg_com, 9 10 output [3:0] o_lab_m_ten, 11 output [3:0] o_lab_m_one, 12 output [3:0] o_lab_s_ten, 13 output [3:0] o_lab_s_one 14); 15 16 wire [7:0] seg_m_ten, seg_m_one, seg_s_ten, seg_s_one; 17 18 reg [10:0] seg_cnt; 19 reg [10:0] m_ten, m_one, s_ten, s_one; 20 reg [10:0] lab_m_ten, lab_m_one, lab_s_ten, lab_s_one; 21 22 wire [7:0] seg_data_req; 23 wire [7:0] seg_com_req; 24 25 assign seg_data = seg_data_req; 26 assign seg_com = seg_com_req; 27 28 always @(posedge clk or posedge rst) begin 29 if (rst) begin 30 seg_cnt <= 0; 31 m_ten <= 0; 32 m_one <= 0; 33 s_ten <= 0; 34 s_one <= 0; 35 end 36 else if (start_btn) begin 37 seg_cnt <= seg_cnt + 1; 38 if (seg_cnt == 9999) begin 39 m_ten <= m_ten + 1; 40 if (m_ten == 59) begin 41 s_ten <= s_ten + 1; 42 if (s_ten == 59) begin 43 seg_cnt <= 0; 44 m_ten <= m_ten + 1; 45 if (m_ten == 10) begin 46 s_ten <= 0; 47 m_one <= m_one + 1; 48 if (m_one == 10) begin 49 seg_cnt <= 0; 50 m_ten <= 0; 51 s_one <= s_one + 1; 52 if (s_one == 10) begin 53 seg_cnt <= 0; 54 m_ten <= 0; 55 m_one <= 0; 56 seg_cnt <= 0; 57 end 58 end 59 end 60 end 61 end 62 end 63 end 64 end 65 66 always @(posedge clk) begin 67 if (rst) seg_data_req <= 8'b1111_1111; 68 else case (s_cnt) 69 0: seg_data_req <= 8'b1111_1111; 70 1: seg_data_req <= 8'b1111_1111; 71 2: seg_data_req <= 8'b1111_1111; 72 3: seg_data_req <= 8'b1111_1111; 73 4: seg_data_req <= 8'b1111_1111; 74 5: seg_data_req <= 8'b1111_1111; 75 6: seg_data_req <= 8'b1111_1111; 76 7: seg_data_req <= 8'b1111_1111; 77 8: seg_data_req <= 8'b1111_1111; 78 9: seg_data_req <= 8'b1111_1111; 79 end 80 end 81 82 always @(posedge clk) begin 83 if (rst) seg_data_req <= 8'b0000_0000; 84 else case (s_cnt) 85 0: seg_data_req <= 8'b0000_0000; 86 1: seg_data_req <= 8'b0000_0000; 87 2: seg_data_req <= 8'b0000_0000; 88 3: seg_data_req <= 8'b0000_0000; 89 4: seg_data_req <= 8'b0000_0000; 90 5: seg_data_req <= 8'b0000_0000; 91 6: seg_data_req <= 8'b0000_0000; 92 7: seg_data_req <= 8'b0000_0000; 93 8: seg_data_req <= 8'b0000_0000; 94 9: seg_data_req <= 8'b0000_0000; 95 end 96 end 97 98 assign o_lab_m_ten = lab_m_ten; 99 assign o_lab_m_one = lab_m_one; 100 assign o_lab_s_ten = lab_s_ten; 101 assign o_lab_s_one = lab_s_one; 102 endmodule </pre>	<pre> 39 always @(posedge clk) begin 40 if (rst) seg_data_req <= 0; 41 else case (scan_cnt) 42 0: seg_data_req <= seg_m_ten; 43 1: seg_data_req <= seg_m_one; 44 2: seg_data_req <= seg_s_ten; 45 3: seg_data_req <= seg_s_one; 46 end 47 end 48 end 49 endmodule </pre>

6. 소리나는 알람 - 추가기능

기존 알람 기능에 청각적 피드백을 추가하여 사용자 인지력을 강화한 확장 모듈이다. 시각적 알람뿐만 아니라 피에조 부저(Piezo Buzzer)를 구동하여 0.5초 간격의 단속음(Beep)을 발생시키는 것이 특징이다. 알람 조건이 충족되면(Current Time == Alarm Time), 시스템은 부저 출력 포트를 제어하여 소리를 송출하며 LED도 함께 점등한다. 특히, 알람이 울리는 도중 사용자가 SW0 스위치를 누르면 활성화된 알람 플레그를 즉시 리셋

[illegible]

날짜 모듈(date.v)은 시스템 클럭과 하루 경과 신호(day_tick)를 입력 받아 현재의 연, 월, 일을 갱신하고 관리한다. 구현 로직은 24시가 지나 day_tick 신호가 인가될 때 일(Day) 데이터를 증가시키는 연산을 수행한다. 이때 각 월(Month)별 마지막 날짜(28, 30, 31일)를 case문을 통해 가변적으로 판단하며, 현재 날짜가 해당 월의 마지막 날짜를 초과하는 경우 일을 1일로 초과화하고 월을 증가시키는 보정 로직을 적용하였다. 또한 12월을 초과하는 연말 시점에서는 연(Year) 데이터를 증가시켜 달력의 연속성을 유지한다. 이외에 스위치 입력을 통해 초기 날짜를 수동으로 설정할 수 있는 FSM(Finite State Machine)이 구현되어 있으며, 최종 날짜 데이터는 시분할 다중화(Time Division Multiplexing) 방식을 거쳐 7-Segment에 표시된다.

[illegible]

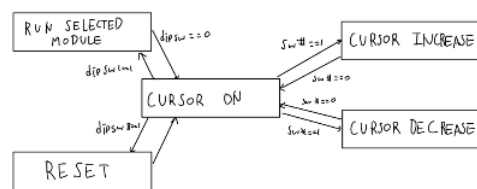
world_time.v

```

1
2  import EIO,
3  import time,
4  import [2:0] int32_zero, int32_one,
5  import [2:0] int32_zero, int32_one,
6  import [2:0] int32_zero, int32_one,
7
8  constant [7:0] int8_data,
9  constant [7:0] int8_data,
10
11  enum [0:0] int8_data,
12  enum [0:0] int8_data,
13
14  enum [0:0] int8_data,
15  enum [0:0] int8_data,
16
17  always #100 begin
18    if (int8_data == 0) begin
19      int8_data <= 1;
20    end else if (int8_data == 1) begin
21      int8_data <= 0;
22    end
23  end
24
25  always #100 begin
26    if (int8_data == 0) begin
27      int8_data <= 1;
28    end else if (int8_data == 1) begin
29      int8_data <= 0;
30    end
31  end
32
33  always #100 begin
34    if (int8_data == 0) begin
35      int8_data <= 1;
36    end else if (int8_data == 1) begin
37      int8_data <= 0;
38    end
39  end
40
41  always #100 begin
42    if (int8_data == 0) begin
43      int8_data <= 1;
44    end else if (int8_data == 1) begin
45      int8_data <= 0;
46    end
47  end
48
49  always #100 begin
50    if (int8_data == 0) begin
51      int8_data <= 1;
52    end else if (int8_data == 1) begin
53      int8_data <= 0;
54    end
55  end
56
57  always #100 begin
58    if (int8_data == 0) begin
59      int8_data <= 1;
60    end else if (int8_data == 1) begin
61      int8_data <= 0;
62    end
63  end
64
65  always #100 begin
66    if (int8_data == 0) begin
67      int8_data <= 1;
68    end else if (int8_data == 1) begin
69      int8_data <= 0;
70    end
71  end
72
73  always #100 begin
74    if (int8_data == 0) begin
75      int8_data <= 1;
76    end else if (int8_data == 1) begin
77      int8_data <= 0;
78    end
79  end
80
81  always #100 begin
82    if (int8_data == 0) begin
83      int8_data <= 1;
84    end else if (int8_data == 1) begin
85      int8_data <= 0;
86    end
87  end
88
89  always #100 begin
90    if (int8_data == 0) begin
91      int8_data <= 1;
92    end else if (int8_data == 1) begin
93      int8_data <= 0;
94    end
95  end
96
97  always #100 begin
98    if (int8_data == 0) begin
99      int8_data <= 1;
100    end else if (int8_data == 1) begin
101      int8_data <= 0;
102    end
103  end
104
105  always #100 begin
106    if (int8_data == 0) begin
107      int8_data <= 1;
108    end else if (int8_data == 1) begin
109      int8_data <= 0;
110    end
111  end
112
113  always #100 begin
114    if (int8_data == 0) begin
115      int8_data <= 1;
116    end else if (int8_data == 1) begin
117      int8_data <= 0;
118    end
119  end
120
121  always #100 begin
122    if (int8_data == 0) begin
123      int8_data <= 1;
124    end else if (int8_data == 1) begin
125      int8_data <= 0;
126    end
127  end
128
129  always #100 begin
130    if (int8_data == 0) begin
131      int8_data <= 1;
132    end else if (int8_data == 1) begin
133      int8_data <= 0;
134    end
135  end
136
137  always #100 begin
138    if (int8_data == 0) begin
139      int8_data <= 1;
140    end else if (int8_data == 1) begin
141      int8_data <= 0;
142    end
143  end
144
145  always #100 begin
146    if (int8_data == 0) begin
147      int8_data <= 1;
148    end else if (int8_data == 1) begin
149      int8_data <= 0;
150    end
151  end
152
153  always #100 begin
154    if (int8_data == 0) begin
155      int8_data <= 1;
156    end else if (int8_data == 1) begin
157      int8_data <= 0;
158    end
159  end
160
161  always #100 begin
162    if (int8_data == 0) begin
163      int8_data <= 1;
164    end else if (int8_data == 1) begin
165      int8_data <= 0;
166    end
167  end
168
169  always #100 begin
170    if (int8_data == 0) begin
171      int8_data <= 1;
172    end else if (int8_data == 1) begin
173      int8_data <= 0;
174    end
175  end
176
177  always #100 begin
178    if (int8_data == 0) begin
179      int8_data <= 1;
180    end else if (int8_data == 1) begin
181      int8_data <= 0;
182    end
183  end
184
185  always #100 begin
186    if (int8_data == 0) begin
187      int8_data <= 1;
188    end else if (int8_data == 1) begin
189      int8_data <= 0;
190    end
191  end
192
193  always #100 begin
194    if (int8_data == 0) begin
195      int8_data <= 1;
196    end else if (int8_data == 1) begin
197      int8_data <= 0;
198    end
199  end
200
201  always #100 begin
202    if (int8_data == 0) begin
203      int8_data <= 1;
204    end else if (int8_data == 1) begin
205      int8_data <= 0;
206    end
207  end
208
209  always #100 begin
210    if (int8_data == 0) begin
211      int8_data <= 1;
212    end else if (int8_data == 1) begin
213      int8_data <= 0;
214    end
215  end
216
217  always #100 begin
218    if (int8_data == 0) begin
219      int8_data <= 1;
220    end else if (int8_data == 1) begin
221      int8_data <= 0;
222    end
223  end
224
225  always #100 begin
226    if (int8_data == 0) begin
227      int8_data <= 1;
228    end else if (int8_data == 1) begin
229      int8_data <= 0;
230    end
231  end
232
233  always #100 begin
234    if (int8_data == 0) begin
235      int8_data <= 1;
236    end else if (int8_data == 1) begin
237      int8_data <= 0;
238    end
239  end
240
241  always #100 begin
242    if (int8_data == 0) begin
243      int8_data <= 1;
244    end else if (int8_data == 1) begin
245      int8_data <= 0;
246    end
247  end
248
249  always #100 begin
250    if (int8_data == 0) begin
251      int8_data <= 1;
252    end else if (int8_data == 1) begin
253      int8_data <= 0;
254    end
255  end
256
257  always #100 begin
258    if (int8_data == 0) begin
259      int8_data <= 1;
260    end else if (int8_data == 1) begin
261      int8_data <= 0;
262    end
263  end
264
265  always #100 begin
266    if (int8_data == 0) begin
267      int8_data <= 1;
268    end else if (int8_data == 1) begin
269      int8_data <= 0;
270    end
271  end
272
273  always #100 begin
274    if (int8_data == 0) begin
275      int8_data <= 1;
276    end else if (int8_data == 1) begin
277      int8_data <= 0;
278    end
279  end
280
281  always #100 begin
282    if (int8_data == 0) begin
283      int8_data <= 1;
284    end else if (int8_data == 1) begin
285      int8_data <= 0;
286    end
287  end
288
289  always #100 begin
290    if (int8_data == 0) begin
291      int8_data <= 1;
292    end else if (int8_data == 1) begin
293      int8_data <= 0;
294    end
295  end
296
297  always #100 begin
298    if (int8_data == 0) begin
299      int8_data <= 1;
300    end else if (int8_data == 1) begin
301      int8_data <= 0;
302    end
303  end
304
305  always #100 begin
306    if (int8_data == 0) begin
307      int8_data <= 1;
308    end else if (int8_data == 1) begin
309      int8_data <= 0;
310    end
311  end
312
313  always #100 begin
314    if (int8_data == 0) begin
315      int8_data <= 1;
316    end else if (int8_data == 1) begin
317      int8_data <= 0;
318    end
319  end
320
321  always #100 begin
322    if (int8_data == 0) begin
323      int8_data <= 1;
324    end else if (int8_data == 1) begin
325      int8_data <= 0;
326    end
327  end
328
329  always #100 begin
330    if (int8_data == 0) begin
331      int8_data <= 1;
332    end else if (int8_data == 1) begin
333      int8_data <= 0;
334    end
335  end
336
337  always #100 begin
338    if (int8_data == 0) begin
339      int8_data <= 1;
340    end else if (int8_data == 1) begin
341      int8_data <= 0;
342    end
343  end
344
345  always #100 begin
346    if (int8_data == 0) begin
347      int8_data <= 1;
348    end else if (int8_data == 1) begin
349      int8_data <= 0;
350    end
351  end
352
353  always #100 begin
354    if (int8_data == 0) begin
355      int8_data <= 1;
356    end else if (int8_data == 1) begin
357      int8_data <= 0;
358    end
359  end
360
361  always #100 begin
362    if (int8_data == 0) begin
363      int8_data <= 1;
364    end else if (int8_data == 1) begin
365      int8_data <= 0;
366    end
367  end
368
369  always #100 begin
370    if (int8_data == 0) begin
371      int8_data <= 1;
372    end else if (int8_data == 1) begin
373      int8_data <= 0;
374    end
375  end
```

9. 추가기능 - menu

메뉴 모듈(menu.v)은 전체 시스템의 동작 모드 선택 및 네비게이션을 담당하는 메인 컨트롤러이다. 사용자로부터 좌우 버튼 입력을 받아 cursor_mode 상태 변수를 0에서 13까지 증감시키며, 이를 통해 총 14가지의 다양한 기능 모드를 순차적으로 탐색한다. 시스템이 특정 기능을 수행 중이지 않은 대기 상태(is_running Low)일 때, LCD 디스플레이에 현재 커서가 가리키는 모드의 명칭(예: "STOPWATCH", "GAME")을 텍스트로 출력하여 직관적인 사용자 인터페이스(UI)를 제공한다. 또한, 마지막 메뉴에서 다음으로 넘어가면 첫 메뉴로, 첫 메뉴에서 이전으로 가면 마지막 메뉴로 순환하는 Wrap-around 로직을 적용하여 탐색의 편의성을 극대화하였다.



(500ms)마다 트리거 신호를 만든다. 이 신호는 서보 모터 컨트롤러로 전달되어 모터의 각도를 0도와 90도로 왕복 운동시키며, 시각적(바늘의 움직임)이고 청각적(모터 구동음)인 박자가 아이디어를 제공한다.

menu.v

10. 추가기능 - timer

타이머 모듈(timer.v)은 설정된 목표 시간으로부터 0이 될 때까지 시간을 차감하는 다운 카운터(Down Counter)이다. 스위치 입력을 통해 시, 분, 초를 순차적으로 설정하고 동작을 시작하면, 내부 타이머를 기반으로 매초마다 값을 감소시킨다. 계수로직은 00초에서 감소 시 분(Minute)을, 00분에서 시(Hour)를 빌려오는(Borrow) 60진법 및 24진법 감산 알고리즘이 하드웨어적으로 구현되어 있다. 카운터 값이 00시 00분 00초에 도달하면 DONE 상태로 전이하며, 이때 내부 카운터를 이용해 led 출력 신호를 주기적으로 토글(Toggle)하여 LED를 점멸시킴으로써 타임아웃을 시각적으로 알린다.

[illegible]

11. 추가기능 - Metronome

메트로놈 모듈은 음악 연주 시 박자를 맞추기 위한 규칙적인 신호를 생성한다. 사용자가 스위치로 설정한 BPM(Beats Per Minute) 값에 따라 주기적으로 신호를 발생시킨다. 예를 들어 60 BPM 설정 시 1초(1000ms)마다, 120 BPM 설정 시 0.5초

```

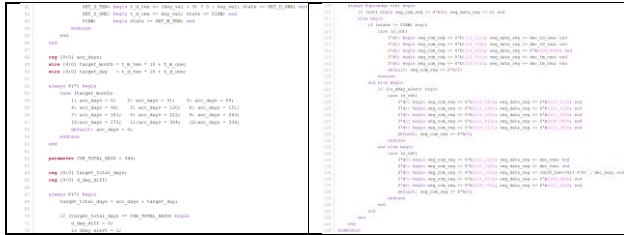
metronome
1 module metronome (
2     input clk,
3     input rst,
4     input [9:0] sw_num,
5
6     output reg servo_pwm,
7     output [7:0] seq_data,
8     output [7:0] seq_cnt,
9 );
10
11 reg [9:0] beat_limit;
12 reg [9:0] display_num;
13
14 always @(posedge clk or posedge rst) begin
15     if (rst) begin
16         beat_limit <= 0;
17         display_num <= 0;
18     end else begin
19         if (sw_num[3:1]) begin
20             beat_limit <= 333;
21             display_num <= 100;
22         end else if (sw_num[2:1]) begin
23             beat_limit <= 500;
24             display_num <= 120;
25         end else if (sw_num[1]) begin
26             beat_limit <= 666;
27             display_num <= 150;
28         end else if (sw_num[0]) begin
29             beat_limit <= 1000;
30             display_num <= 60;
31         end else begin
32             beat_limit <= 0;
33             display_num <= 0;
34         end
35     end
36 end
37
38 reg [9:0] time_cnt;
39 reg direction;
40 reg [9:0] time_req;
41 reg direction;
42
43 always @(posedge clk or posedge rst) begin
44     if (rst) begin
45         time_cnt <= 0;
46         direction <= 0;
47     end else if (beat_limit > 0) begin
48         if (time_cnt >= beat_limit) begin
49             time_cnt <= 0;
50             direction <= ~direction;
51         end else begin
52             time_cnt <= time_cnt + 1;
53         end
54     end else begin
55         time_cnt <= 0;
56         direction <= 0;
57     end
58 end
59
60 seq [4:0] pwm_cnt;
61 always @(posedge clk or posedge rst) begin
62     if (rst) begin
63         pwm_cnt <= 0;
64         servo_pwm <= 0;
65     end else begin
66         if (pwm_cnt >= 150) pwm_cnt <= 0;
67         else pwm_cnt <= pwm_cnt + 1;
68     end
69     if (direction == 0) begin
70         servo_pwm <= (pwm_cnt < 1) ? 1 : 0;
71     end else begin
72         servo_pwm <= (pwm_cnt < 2) ? 1 : 0;
73     end
74 end
75
76 wire [7:0] dec_hundred, dec_ten, dec_one;
77
78 wire [3:0] b_hundred = display_num / 100;
79 wire [3:0] b_ten = (display_num % 100) / 10;
80 wire [3:0] b_one = display_num % 10;
81
82 seq_decode u_b (dec_hundred, dec_hundreds);
83 seq_decode u_t (b_ten, dec_tens);
84 seq_decode u_o (b_one, dec_ones);
85
86 reg [2:0] s_cnt;
87 reg [7:0] seq_data_req, seq_cnt_req;
88 assign seq_data = seq_data_req;
89 assign seq_cnt = seq_cnt_req;
90
91 always @(posedge clk) begin
92     if (rst) s_cnt <= 0;
93     else if (s_cnt >= 8) s_cnt <= 0;
94     else s_cnt <= s_cnt + 1;
95 end
96
97 always @(posedge clk) begin
98     if (rst) seq_data_req <= 0;
99     if (seq_data_req >= 8) seq_data_req <= 0;
100     else seq_data_req <= seq_data_req + 1;
101 end
102
103 always @(posedge clk) begin
104     if (rst) seq_cnt_req <= 0;
105     if (seq_cnt_req >= 8) seq_cnt_req <= 0;
106     else seq_cnt_req <= seq_cnt_req + 1;
107 end
108
109 endmodule

```

12. 추가기능 - 디데이 계산기

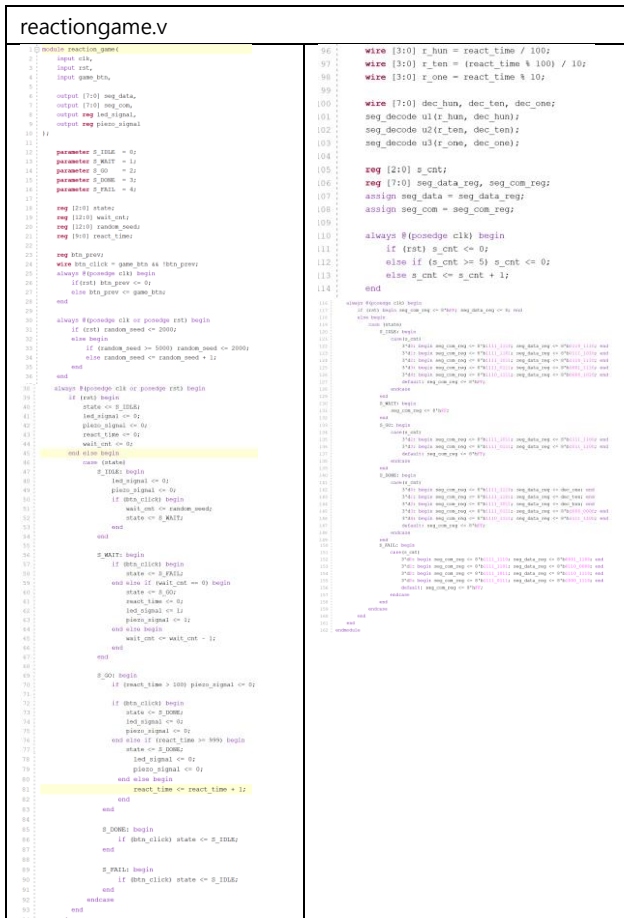
디데이 모듈(dday.v)은 룩업 테이블(Lookup Table)을 활용하여 목표 날짜와 현재 날짜(2025-12-12) 사이의 남은 일수를 계산한다. 연산 로직은 월/일 형식의 날짜를 1월 1일 기준의 총 경과 일수(Day Count)로 환산한 뒤, 두 값의 차이를 구하는 방식으로 구현되었다. 만약 목표 날짜가 현재 날짜보다 과거(내년)일 경우 365일을 더하여 보정하며, 두 날짜가 일치하는 당일에는 is_dday_alert 신호를 활성화한다. 이 신호가 발생하면 7-Segment 디스플레이에 "d - d A Y"라는 문구를 출력하고, 동시에 top_system의 제어 하에 모든 LED를 점등하고 피에조 부저를 구동시켜 사용자가 설정한 날짜가 도래했음을 시청각적으로 알린다.

[illegible]



13. 추가기능 - 반응속도 테스트

반응속도 테스트 모듈(reaction_game.v)은 무작위적인 시각 및 청각 자극에 대한 사용자의 순발력을 정량적으로 측정한다. 게임이 시작되면 예측 불가능한 대기 시간(Random Delay)이 지난 후 LED 점등과 동시에 피에조 부저음이 발생한다. 난수 생성은 2000ms에서 5000ms 사이를 고속으로 왕복하는 프리러닝 카운터(Free-running Counter) 값을 사용자의 시작 버튼 입력 시점에 샘플링(Sampling)하는 방식을 적용하여 구현하였다. 시스템은 자극 발생 시점부터 사용자의 버튼 입력 시점까지의 경과 시간을 1ms 단위로 정밀 측정하여 7-Segment에 표시한다. 또한, 신호가 발생하기 전에 버튼을 미리 누르는 부정 출발(False Start)을 감지하여 즉시 실패(FAIL) 처리하는 로직을 포함함으로써 게임의 공정성을 확보하였다.



14. 추가기능 - 동요 노래

음악 연주 모듈(music.v)은 미리 프로그래밍된 악보 데이터를 기반으로 5곡의 동요를 피에조 부저로 재생하는 기능을 수행한

다. 스위치 입력을 통해 비행기, 학교중, 나비야 등의 곡을 선택하거나 재생을 정지할 수 있으며, 선택된 곡 번호는 "PLAY - n" 형식으로 7-Segment에 시각화된다. 내부 동작은 tone_cnt를 이용한 클럭 분주로 각 음계의 고유 주파수(구형파)를 생성하는 톤 제너레이터(Tone Generator)와, beat_cnt를 통해 음표의 지속 시간(Tempo)을 제어하는 시퀀서(Sequencer) 구조로 이루어져 있다. 악보 데이터는 case 문을 활용한 상태 머신 내에 록업 테이블 형태로 구현되어 있어, 음표 인덱스(note_idx)가 증가함에 따라 정해진 멜로디 라인을 순차적으로 출력한다.



15. 추가기능 - 피아노

피아노 모듈(piano.v)은 FPGA 보드의 스위치를 건반으로 활용하여 사용자가 직접 멜로디를 연주할 수 있는 실시간 인터페이스를 제공한다. 50MHz 시스템 클럭을 기반으로 정밀하게 계산된 도(C4)부터 높은 도(C5)까지의 주파수 주기가 SW0~SW7 스위치에 매핑되어 있어, 입력 즉시 해당 음계의 구형파를 피에조 부저로 출력한다. 시각적 피드백을 위해 스위치 입력 시 7-Segment 디스플레이에 해당 게이름(예: 'd' for Do, 'r' for Re)을 패턴으로 표시하여 직관성을 확보하였다. 단음(Monophonic) 연주 방식이므로 화음은 지원하지 않으나, if-else if 구조의 우선 순위 인코더(Priority Encoder) 로직을 적용하여 다중 입력 발생 시 낮은 음(낮은 번호의 스위치)이 우선적으로 출력되도록 설계하여 신호 충돌을 방지하였다.

```

piano.v
1 module piano
2   input clk,
3   input rst,
4   input [15:0] pw_mem,
5
6   output reg piano_out,
7   output [17:0] seg_data,
8   output [17:0] seg_cen
9
10  32
11
12  parameter C4 = 85542;
13  parameter D4 = 95132;
14  parameter B4 = 10482;
15  parameter G4 = 13362;
16  parameter A4 = 50182;
17  parameter F4 = 50612;
18  parameter C5 = 47772;
19
20  reg [15:0] counter;
21  reg [17:0] period;
22  reg [17:0] note_display;
23
24  always @(*) begin
25    if (sw_mem[0]) begin period = C4; note_display = 8'b000_000; end
26    else if (sw_mem[1]) begin period = D4; note_display = 8'b000_111; end
27    else if (sw_mem[2]) begin period = B4; note_display = 8'b001_011; end
28    else if (sw_mem[3]) begin period = F4; note_display = 8'b001_110; end
29    else if (sw_mem[4]) begin period = A4; note_display = 8'b010_110; end
30    else if (sw_mem[5]) begin period = B4; note_display = 8'b011_110; end
31    else if (sw_mem[6]) begin period = C5; note_display = 8'b100_000; end
32    else if (sw_mem[7]) begin period = D5; note_display = 8'b100_000; end
33    else begin period = B4; note_display = 8'b011_000; end
34  end
35
36  always @(posedge clk or posedge rst) begin
37    if (rst) begin
38      counter <= 0;
39      piano_out <= 0;
40    end else begin
41      if (period == 0) begin
42        counter <= 0;
43        piano_out <= 0;
44      end else begin
45        if (counter == period) begin
46          counter <= 0;
47          piano_out <= ~piano_out;
48        end else begin
49          counter <= counter + 1;
50        end
51      end
52    end
53  end
54
55  assign seg_cen = 8'b100_000;
56  assign seg_data = note_display;
57 endmodule

```

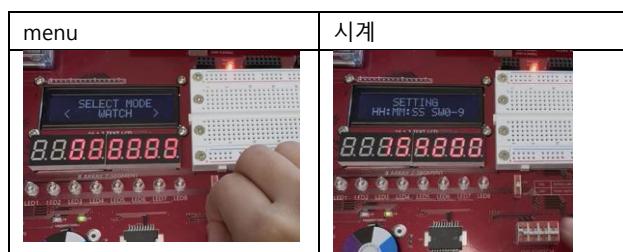
5. 구현 결과(Implementation Results)

a. 준비물

노트북, combo2

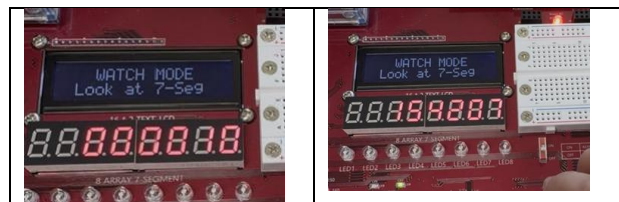
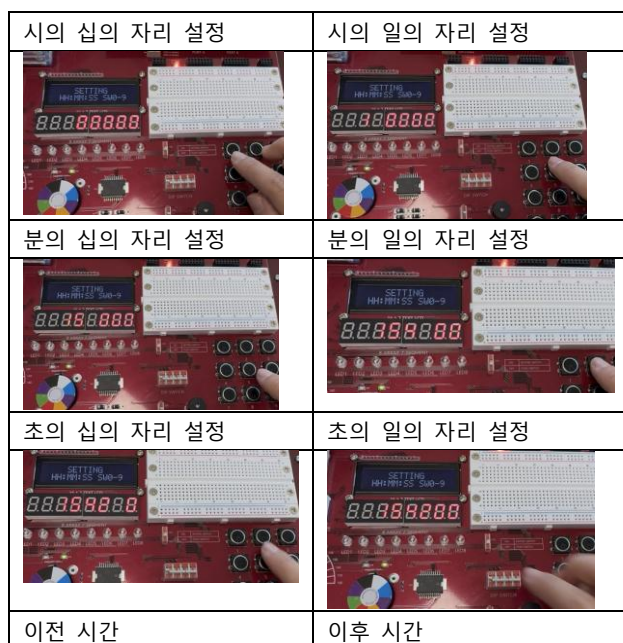
b. 기능변 동작 사진

1. 전자시계



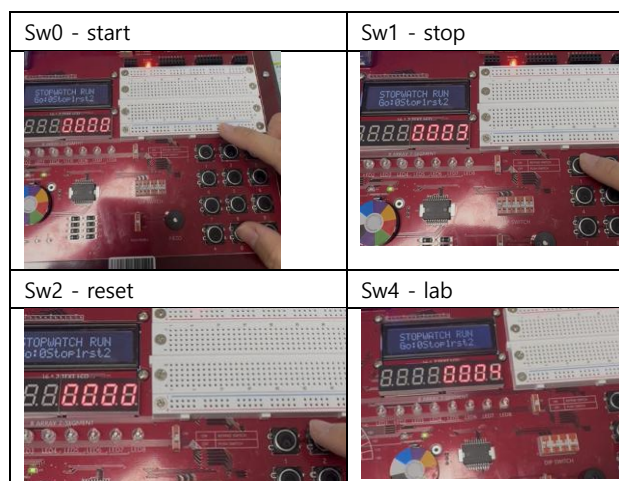
menu에서 dip스위치1을 올리면, 위와 같이 전자시계가 나타난다.

2. 시간 설정



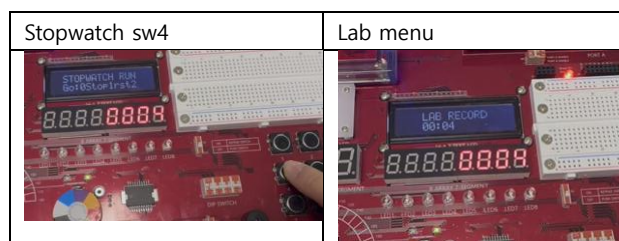
우측의 키패드 형태의 스위치를 통해 시의 십의 자리 일의 자리, 분의 십의 자리, 초의 십의 자리, 초의 일의 자리순으로 키패드를 눌러 설정한다. 이후, watch 메뉴에 들어가면, 설정한 시간으로 세팅됨을 확인할 수 있다.

3. Stopwatch



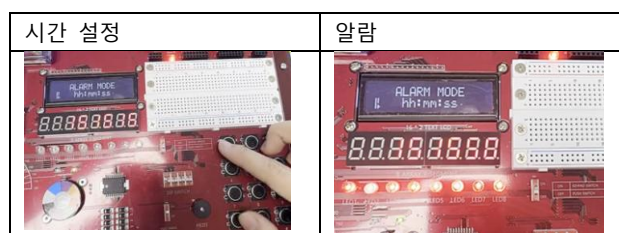
Sw0을 누르면 stpwatch가 시작된다. Sw1을 누르면 Stopwatch가 멈추고, sw2를 누르면 초기화되며, sw4를 누르면 lab 저장된다. 물론, 해당 stopwatch에서는 표기되지 않는다.

4. LAB확인기능



Stopwatch에서 sw4를 눌러 원하는 시간을 저장한다. 이후, lab을 확인하는 메뉴에 가면 저장한 값을 확인할 수 있다.

5. 알람



알람을 15:15:50에 맞춰두었다. 알람을 우측의 스위치를 통해 시의 십의 자리 일의 자리, 분의 십의 자리, 초의 십의 자리,

초의 일의 자리순으로 키패드를 눌러 설정한다. Sw0을 누르기 전까지 led1~led8은 켜져 알람이 울렸음을 나타낸다.

6. 소리나는 알람

시간 설정	알람
	

알람 설정 방법은 기본 알람과 동일하다. 15:16:20초로 설정한 실험이다. 지정된 시간이 되면 소리가 나며, led1도 함께 켜진다. Sw0을 누르면 알람이 종료된다.

7. Date

메뉴	날짜 표기
	

날짜 메뉴를 dip스위치1을 올려 들어가면, 날짜가 나온다. 시계 설정과 같은 방법으로 날짜를 설정할 수 있다.

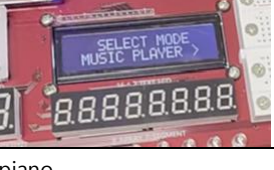
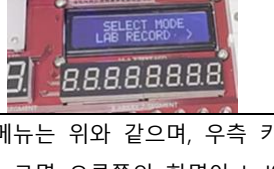
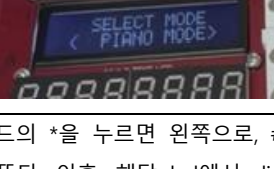
8. World clock

설정시간(한국)	세계표준시
	

왼쪽은 이전에 설정한 한국시간이다. 이후, worldclock메뉴에 들어가면, 세계표준시를 확인할 수 있다.

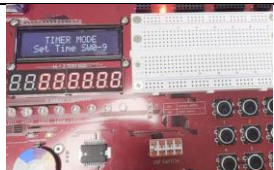
9. Menu

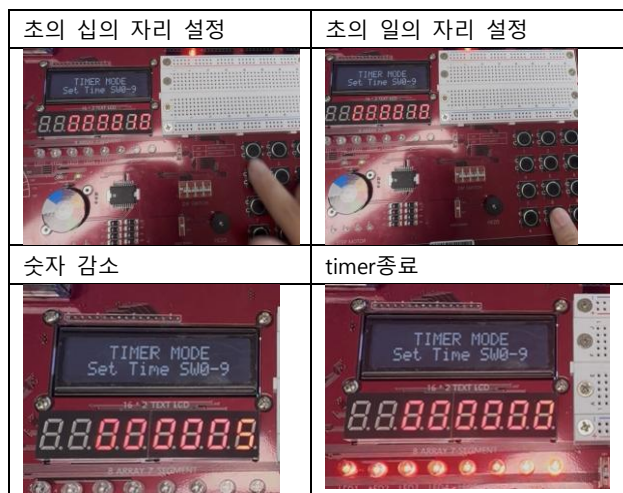
watch	stopwatch
	
Timer	Setting

	
world	Alarm
	
date	Sound alarm
	
metonome	d-day
	
Reaction game	music
	
Lab record	piano
	

메뉴는 위와 같으며, 우측 키패드의 *을 누르면 왼쪽으로, #을 누르면 오른쪽의 화면이 lcd에 뜬다. 이후, 해당 lcd에서 dip스위치1을 올리면, 메뉴에 해당하는 내용이 실행되었다.

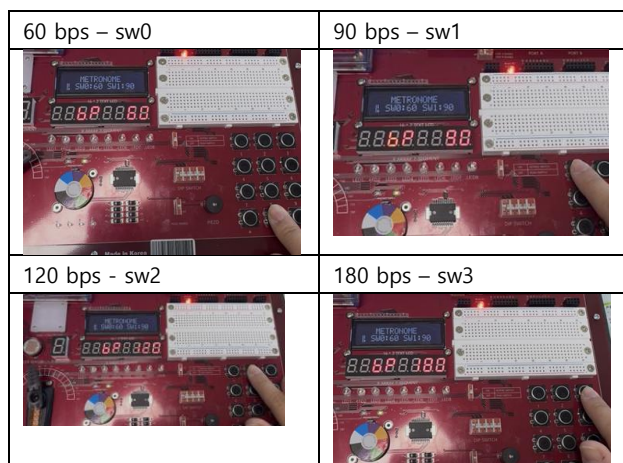
10. Timer

시의 십의 자리 설정	시의 일의 자리 설정
	
분의 십의 자리 설정	분의 일의 자리 설정
	



타이머는 우측의 스위치를 통해 시의 십의 자리 일의 자리, 분의 십의 자리, 초의 십의 자리, 초의 일의 자리순으로 키패드를 눌러 설정한다. 마지막 초의 일의자리를 입력하고 나면, 바로 타이머가 시작되며, 예시 실험으로는 10초 카운트 다운을 해보았다. 0초가 남으면, led1~led8에 불이 깜빡거린다.

11. Metronome



Servo motor를 이용하여 metronme을 만들었다. Sw0을 누르면 60 bps가, sw1은 90bps, sw2는 120bps, sw3는 120 bps에 맞추어 servo motor가 작동하였다.

12. 디데이 계산기



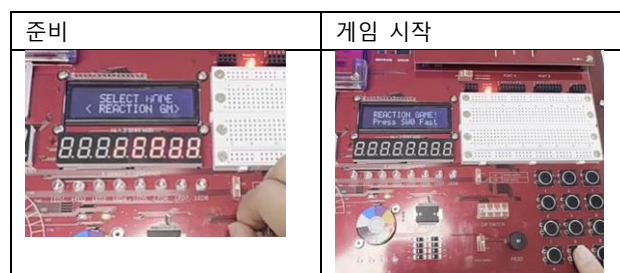
우측의 키패드를 이용하여 원하는 날짜를 선택한다.

디데이 계산 결과



12/22은 12/12의 10일 후이므로 위와 같이 세그먼트에 나타난다.

13. 반응속도 테스트

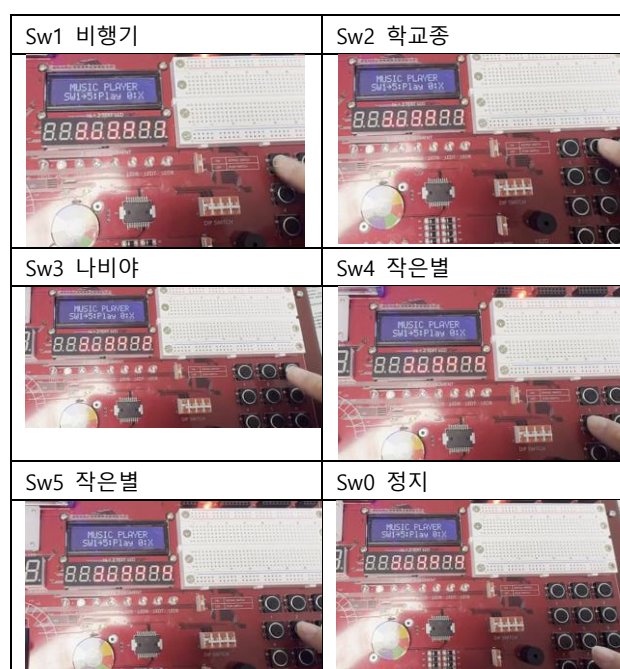


ready라는 문구가 뜬다. 게임을 진행할 준비가 되면 sw0을 누른다. 이후, 게임이 시작되면 lcd가 오른쪽과 같이 바뀐다.



이후, 랜덤하게 led와 소리가 나타나면, 빠르게 sw0을 누른다. 이후, led, 소리가 나타난 지 몇 초만에 반응하였는지 결과가 나타난다.

14. 동요 노래



각 버튼에 해당하는 음악이 나오며, sw0을 통해 정지시킬 수 있다.

15. 피아노



Sw1~sw8을 누르면 피아노와 같이 소리가 난다. Sw1이 가장 낮은 소리이며 sw의 숫자가 높을수록 음이 올라간다.

6. 결론 및 고찰(Conclusion & Discssion)

a. 설계 요약

디지털 시계 모듈(watch.v)은 1kHz 클럭을 입력받아 시, 분, 초를 계수하고 이를 7-Segment에 출력한다. 내부 로직은 0부터 999까지 증가하는 ms_cnt를 기준으로 1초 신호를 생성하며, 이를 통해 초(60진법), 분(60진법), 시(24진법) 카운터가 순차적으로 증가하는 계층적 구조를 갖는다. 시간이 23시 59분 59초를 넘어가면 00시로 초기화되고 동시에 날짜 변경을 위한 신호를 발생시킨다. 또한, 시간 설정 모듈(setting.v)에서 load_time 신호가 입력되면 사용자가 설정한 값을 현재 카운터 레지스터에 즉시 반영하여 시간을 수정하도록 구현하였다. 디스플레이 출력부는 3비트 카운터를 이용한 다이내믹 스캐닝 방식을 적용하여, 6개의 자릿수를 고속으로 순환 점등시킴으로써 데이터를 표시한다. 이와 연계된 날짜 모듈(date.v)은 현재 2025년 12월 12일의 고정 데이터를 출력하도록 설계되었으며, 세계 시간 모듈(world_time.v)은 한국 표준시(KST)에서 9시간을 감산하되 연더플로우를 방지하는 모듈러 연산을 적용하여 런던(UTC+0) 기준 시간을 실시간으로 변환하여 제공한다.

스톱워치 모듈(stopwatch.v)은 1ms 단위의 내부 카운터를 기반으로 정밀 시간 계수를 수행하며, 랩 타임(Lab Time) 기능을 위해 현재 시간을 별도 레지스터에 래치(Latch)하여 저장한다. 랩 기록(Lab Record) 모드에서는 이 저장된 정적 데이터를 출력하여 기록을 조회할 수 있다. 알람 모듈(alarm.v)은 설정 시각과 현재 시각(시/분/초)이 일치할 경우 알람 활성화 플래그를 래치하여 LED를 점등시키며, 소리 알람 모듈(alarmpiezo.v)을 통해 0.5초 간격의 단속음(Beep)을 출력한다.

메트로놈 모듈(metronome.v)은 설정된 BPM에 맞춰 20ms 주기의 PWM 신호를 1ms와 2ms로 교차 변조하여 서보 모터를 왕복 운동시키며 박자를 제공한다. 피아노 모듈(piano.v)은 50MHz 시스템 클럭을 기반으로 각 스위치에 매핑된 음계 주파수(C4~C5)를 생성하는 실시간 연주 기능을 구현하였다. 반응 속도 테스트 모듈(reaction_game.v)은 프리러닝 카운터 샘플링 방식으로 랜덤 딜레이를 생성하여 순발력을 측정한다. 모든 기능은 메뉴 모듈(menu.v)의 랩 어라운드(Wrap-around) 로직을

통해 총 14가지 모드 사이를 탐색하며 실행된다.

b. 문제점 및 해결 과정(troubleshooting)

1. 클럭 및 타이밍 불일치 문제

프로젝트 초기, 메인 클럭(50MHz)과 시간 계수 모듈이 요구하는 1kHz 타이밍 간의 불일치 문제가 발생하였다. 다수의 모듈(Watch, Stopwatch, Timer)이 1ms 단위의 정확도를 필요로 했으나, 피아노 모듈(piano.v)은 50MHz 원본 클럭을 기준으로 주파수를 계산해야 하는 충돌 지점이 있었다. 이를 해결하기 위해, Top Module에서 별도의 클럭 분주(Clock Division)를 구현하는 대신, 모든 시간 계수 모듈이 1kHz 클럭 틱을 받아 내부 카운터(ms_cnt)를 통해 필요한 시간 단위(1초, 1분 등)를 자체적으로 생성하도록 설계 구조를 통일하였다. 이로써 piano 모듈은 고속 클럭을 그대로 사용하여 음계의 정밀도를 확보하고, 나머지 모듈들은 1ms 단위의 정확한 시간 계수를 병행할 수 있게 되었다.

2. 동시 입력 및 엣지 검출 안정화

물리적인 스위치 및 버튼(SW0~SW9, BTN* / BTN#) 입력 시 채터링(Chattering) 현상과 부정확한 입력 인식 문제가 발생하였다. 특히 타이머/알람 설정 중 여러 스위치가 동시에 눌릴 경우, 잘못된 숫자가 입력되거나 상태 머신이 불안정하게 천이하는 현상이 관찰되었다. 이 문제는 디바운싱(Debouncing) 로직과 엣지 검출(Edge Detection) 방식을 결합하여 해결하였다. 모든 입력 버튼 및 스위치에 대해 1클럭 이전 값(sw_prev, btn_prev)을 저장한 뒤, 현재 입력과의 비교를 통해 상승 엣지(sw_rise, btn_click) 순간만을 정확히 인식하도록 구현함으로써, 시스템의 반응성과 안정성을 동시에 확보하였다.

3. 출력 데이터 충돌 및 모드 전환 문제

총 14가지 모듈이 7-Segment, LED, Piezo 부저라는 제한된 출력 포트를 공유하면서 모드 전환 시 출력 데이터가 섞이는 충돌 문제가 있었다. 특히 Piezo 부저는 알람, D-Day, 반응 게임, 음악, 피아노 등 여러 모듈에서 독립적으로 구동 신호를 요구했다. 이 문제를 해결하기 위해, Top Module에 current_cursor 값을 기준으로 동작하는 대규모 멀티플렉서(MUX) 구조를 적용하였다. 실행 중인 모드(is_app_running High)가 결정되면, 해당 모듈의 출력 신호만 선택적으로 최종 출력 포트에 라우팅하고 나머지 모듈의 신호는 차단함으로써, 데이터 충돌 없이 매끄럽고 안정적인 출력을 보장하였다.

4. 유한 상태 머신의 비정상적 상태 천이 문제

초기 설계 단계에서 메인 메뉴 탐색 FSM() 및 세부 시간 설정 FSM()이 사용자의 빠른 입력이나 불안정한 버튼 신호에 의해 의도하지 않은 상태로 천이하거나 멈추는 문제가 발생하였

다. 이는 FSM의 상태 천이 조건에 단순 버튼 입력 신호 (sw_num)를 사용했기 때문이었다. 이 문제를 해결하기 위해, 모든 상태 천이 조건에 디바운싱(Debouncing) 처리된 '단일 클릭' 신호만을 사용하도록 로직을 수정하였다. 특히, 시간 설정 FSM에서는 '값 증가' 키와 '다음 자리로 이동' 키의 역할을 명확히 분리하고, 각 상태(예: SET_H_TEN) 내에서 다음 상태로의 천이 조건이 다른 모든 조건을 압도하는(Mutually Exclusive) 방식으로 우선순위를 조정하여, FSM이 예측 가능하고 안정적으로 동작하도록 보장하였다.

5. 7-segment 동적 스캐닝 출력 품질과

6자리의 7-Segment를 하나의 데이터 버스와 6개의 공통 핀을 이용해 구동하는 동적 스캐닝(Dynamic Scanning) 방식 채택 후, 스캐닝 주기의 미세한 불일치로 인해 화면 깜빡임(Flicker) 또는 이전 데이터가 겹쳐 보이는 잔상(Ghosting) 현상이 발생하였다. 이는 사용자 가독성을 저해하는 주요 문제였다. 해결책으로, 먼저 스캐닝 속도를 약 250Hz 내외로 유지하는 최적의 분주율을 계산하여 깜빡임 문제를 해결하였다. 더 중요하게는, 잔상 현상을 제거하기 위해 스캐닝 카운터가 다음 자리로 넘어가기 직전에 해당 자리의 공통 핀 출력을 짧은 시간(수 마이크로초) 동안 강제로 끄는(Blanking) 로직을 추가하였다. 이 블랭킹 시간을 통해 이전 데이터가 완전히 소거된 후 새로운 데이터가 출력되도록 함으로써, 잔상 없는 깨끗하고 안정적인 디스플레이 출력을 확보하였다.

c. 개선 사항 및 제한

1. 하드웨어 아키텍처 및 클럭 관리 개선

Top Module 내부에 전용 클럭 분주기 모듈을 설계하여 50MHz 입력을 받아 1kHz 주파수 신호를 명시적으로 생성해야 한다. 이후 각 모듈에는 필요한 클럭(예: Watch에는 1kHz, Piano에는 50MHz)만 개별적으로 연결하여 시스템의 모듈성과 안정성을 높여야 한다

2. 핵심 기능의 독립성 및 정확성 확보

현재 Date 모듈은 고정 날짜만을 출력하고 있으므로, Watch 모듈에서 발생하는 자정 신호를 받아 일(Day) 카운터를 증분하고, 월의 길이 판단 및 윤년 계산 로직을 구현하여 시스템의 자가 유지(Self-sustaining) 기능을 완성해야 한다.

3. 추가기능 수정사항 - 알람

소리 알람의 기능 중 소리 부분에서 목표했던 바는 알람 음악을 설정 할 수 있는 기능을 만드는 것이었다. 하지만, clock 분주 설정에 의하여, 현재

music에서 재생하는 주파수와 현재 watch.v의 주파수가 일치하지 않는다. 분주를 다시 하여 여러 차례 시도하였으나, 성공하지는 못했다.

4. 추가 기능 심화

현재 구현된 피아노 모듈은 피에조 부저를 통해 단순히 실시간 입력된 음계를 출력하는 단음 생성(Monophonic Generation) 기능에 국한되어 있다. 이에 대한 기능 고도화 방안으로, 사용자가 연주한 멜로디 패턴을 저장하여 이를 디지털 시계의 알람음으로 활용하는 커스텀 알람 및 레코딩 기능을 제안한다. 구체적인 구현 로직은 다음과 같다. 우선 키패드 스캐닝을 통해 인가된 노트(Note) 데이터를 감지하여 순차적으로 레지스터에 저장한다. 동시에 watch.v 모듈의 시스템 타이머와 동기화하여 각 노트의 입력 시점과 지속 시간(Duration)을 측정해 델타 타임(Delta Time) 정보를 함께 기록한다. 이를 통해 재생 시 단순한 음의 나열이 아닌, 사용자의 연주 리듬까지 완벽하게 복원하여 알람 동작 시 재생하는 시스템을 구현할 예정이다.

7. 부록(Appendix)

- segdecode.

segdecode.v	
<pre> 1 module seg_decode (data_in, seg_data); 2 3 input [3:0] data_in; 4 output [7:0] seg_data; 5 6 reg [7:0] seg_data; 7 8 always @(data_in) 9 begin 10 case (data_in) 11 4'b0000 : seg_data = 8'b1111_1100; // 0 12 4'b0001 : seg_data = 8'b1101_0000; // 1 13 4'b0010 : seg_data = 8'b1101_1001; // 2 14 4'b0011 : seg_data = 8'b1111_0010; // 3 15 4'b0100 : seg_data = 8'b1010_0110; // 4 16 4'b0101 : seg_data = 8'b1011_0110; // 5 17 4'b0110 : seg_data = 8'b1011_1101; // 6 18 4'b0111 : seg_data = 8'b1000_1110; // 7 19 20 default: seg_data = 8'b1111_1111; // 8 21 endcase 22 end 23 endmodule </pre>	<pre> 17 4'b1111 : seg_data = 8'b1110_0000; // 9 18 4'b1000 : seg_data = 8'b1111_1110; // 10 19 4'b1001 : seg_data = 8'b1111_0110; // 11 20 4'b1010 : seg_data = 8'b1110_1110; // 12 21 4'b1011 : seg_data = 8'b1011_1110; // 13 22 4'b1100 : seg_data = 8'b1001_1100; // 14 23 4'b1101 : seg_data = 8'b1011_1010; // 15 24 4'b1110 : seg_data = 8'b1001_1110; // 16 25 4'b1111 : seg_data = 8'b1000_1110; // 17 26 27 28 endcase 29 endmodule </pre>

- textlcd.v

textlcd.v	
<pre> 1 module textlcd 2 (3 input clk, 4 input [15:0] line1_in, 5 input [15:0] line2_in, 6 output [7:0] lcd_data, 7 output [7:0] lcd_pos, 8); 9 10 reg [7:0] lcd_data; 11 reg [7:0] lcd_pos; 12 13 reg [15:0] status; 14 parameter delay = 5, function_sel = 1, entry_mode = 2, disp_mode = 3, 15 line1 = 0, line2 = 1; 16 17 always @(clk) 18 begin 19 if (function_sel == 1) 20 begin 21 if (entry_mode == 1) 22 begin 23 if (disp_mode == 1) 24 begin 25 if (line1 == 0) 26 begin 27 lcd_data = line1_in; 28 lcd_pos = 0; 29 end 30 else if (line1 == 1) 31 begin 32 lcd_data = line2_in; 33 lcd_pos = 1; 34 end 35 end 36 else if (disp_mode == 2) 37 begin 38 if (line1 == 0) 39 begin 40 lcd_data = line1_in; 41 lcd_pos = 0; 42 end 43 else if (line1 == 1) 44 begin 45 lcd_data = line2_in; 46 lcd_pos = 1; 47 end 48 end 49 end 50 else if (function_sel == 2) 51 begin 52 if (entry_mode == 1) 53 begin 54 if (disp_mode == 1) 55 begin 56 lcd_data = line1_in; 57 lcd_pos = 0; 58 end 59 else if (disp_mode == 2) 60 begin 61 lcd_data = line2_in; 62 lcd_pos = 1; 63 end 64 end 65 end 66 end 67 end 68 endmodule </pre>	<pre> 69 always @(function_sel) 70 begin 71 if (function_sel == 1) 72 begin 73 if (entry_mode == 1) 74 begin 75 if (disp_mode == 1) 76 begin 77 if (line1 == 0) 78 begin 79 lcd_data = line1_in; 80 lcd_pos = 0; 81 end 82 else if (line1 == 1) 83 begin 84 lcd_data = line2_in; 85 lcd_pos = 1; 86 end 87 end 88 else if (disp_mode == 2) 89 begin 90 if (line1 == 0) 91 begin 92 lcd_data = line1_in; 93 lcd_pos = 0; 94 end 95 else if (disp_mode == 2) 96 begin 97 lcd_data = line2_in; 98 lcd_pos = 1; 99 end 100 end 101 end 102 else if (function_sel == 2) 103 begin 104 if (entry_mode == 1) 105 begin 106 if (disp_mode == 1) 107 begin 108 lcd_data = line1_in; 109 lcd_pos = 0; 110 end 111 else if (disp_mode == 2) 112 begin 113 lcd_data = line2_in; 114 lcd_pos = 1; 115 end 116 end 117 end 118 end 119 endmodule </pre>

8. 참고문헌(Reference)

- 전자전기컴퓨터설계실습II_2.1 디지털 회로 설계.pdf
- 전자전기컴퓨터설계실습II-2.2_논리 게이트 구현.pdf
- 전자전기컴퓨터설계실습II_3.1 기초 조합 논리 회로 설계(이론).pdf
- 전자전기컴퓨터설계실습II_3.2_기초 조합 논리 회로 설계(실습).pdf
- 전자전기컴퓨터설계실습II_4.1 고급 조합 논리 회로 설계(이론).pdf
- 전자전기컴퓨터설계실습II_4.2_기초 조합 논리 회로 설계(실습).pdf
- 전자전기컴퓨터설계실습II_5.1 연산 논리 회로 설계 (이론).pdf
- 전자전기컴퓨터설계실습II_5.2_연산 논리 회로 설계 (실습).pdf
- 전자전기컴퓨터설계실습II_6.1_기초 순차 논리 회로 설계(이론).pdf
- 전자전기컴퓨터설계실습II_6.2_기초 순차 논리 회로 설계(실습).pdf
- 전자전기컴퓨터설계실습II_7.1_고급 순차 논리 회로 설계(이론).pdf
- 전자전기컴퓨터설계실습II_7.2_고급 순차 논리 회로 설계(실습).pdf
- 전자전기컴퓨터설계실습II_8.1_FSM 설계(이론).pdf
- 전자전기컴퓨터설계실습II_8.2_FSM 설계(실습).pdf
- 전자전기컴퓨터설계실습II_9.1_CHARACTER LCD 제어 (실습).pdf
- Digital Design With an Introduction to the Verilog HDL, VHDL, and SystemVerilog 6th edition, M. Morris Mano